

Reproducing HyperTurtle: Accelerating Nested Virtualization with Hyperupcalls

Manoj Khatri

April 2026

Abstract

This report reproduces the primary results of the USENIX ATC 2025 paper *Accelerating Nested Virtualization with HyperTurtle* on two CloudLab c220g5 nodes. HyperTurtle offloads L1 hypervisor logic into L0 as eBPF programs across three subsystems: EPT fault handling, networking, and profiling. All three subsystems ran successfully. On EPT, HyperTurtle reduced median L2 fault latency from $23.43\ \mu\text{s}$ to $5.55\ \mu\text{s}$, a $4.22\times$ speedup, and improved Kata startup time by 20–33% across six images. Networking and profiling also reproduce the paper’s main claims: HyperTurtle network functions stay close to the fast path and substantially outperform the nested baseline, and profiling overhead at 1 kHz is substantially lower than with conventional L1 profiling.

1. Introduction

Nested virtualization lets a guest VM run its own hypervisor and manage further VMs, which is the basis of VM-based container deployments such as Kata Containers, but its main weakness is performance. Every L2 event requiring hypervisor intervention must pass through both the guest hypervisor (L1) and the bare-metal hypervisor (L0), creating multiple world switches per event. The paper quantifies this cost: each world switch takes roughly $1\ \mu\text{s}$ and accounts for about 33% of total vm-exit handling time. On the EPT fault path a single L2 fault requires six world switches against two in the non-nested case, producing more than a $5\times$ average-latency increase and a substantial increase in Kata container boot time.

HyperTurtle reduces this overhead without removing L1 from control of L2. Its key mechanism is the *hyperupcall*: a small eBPF program encapsulating a fragment of L1 hypervisor logic that executes in the L0 context when an L2 vm-exit occurs, eliminating the world switches for that event. Because hyperupcalls run through the kernel’s existing eBPF verifier, they require only modest changes to L0 KVM and QEMU and no modifications to L2. The paper applies this to three subsystems: EPT fault handling, L1-controlled network I/O, and application profiling, and reports substantial gains in all three.

This report reproduces those results using the authors’ public repository (<https://github.com/acsl-technion/hyperturtle>) on two CloudLab c220g5 nodes in the same Intel Skylake-SP CPU family as the paper’s hardware. The target experiments are EPT fault latency and Kata startup, network performance, and profiling overhead. The repository contains only build instructions. The authors provided a VM launch script on request, which required further modification. All remaining evaluation infrastructure was written from scratch, and substantial code-level fixes were required before any experiment could run. The EPT path in particular required fixing several prototype bugs and making the boot sequence deterministic before results matched the paper.

2. Experimental Setup

2.1. Hardware

I ran the evaluation on two CloudLab c220g5 nodes (Figure 1). Hardware differences from the paper are summarised in Table 1. Both CPU models belong to the Skylake-SP family with 46-bit physical addressing, making this a reasonable platform for trend-level comparison. One node was the nested-virtualization host running L0, L1, and L2, and the second acted as the benchmark client for networking experiments.

Table 1: Hardware differences between the paper and this setup (common factors: Skylake-SP family, 46-bit physical addressing, SMT off).

Component	Paper	This setup
CPU model	2× Xeon Silver 4216	2× Xeon Silver 4114
Cores per NUMA node	16	10
CPU base clock	2.1 GHz	2.2 GHz
L3 cache	22 MB	13.75 MB
RAM	512 GiB	~192 GiB
NIC	Nvidia MT27710 (ConnectX-4)	Intel X520/X540
NIC link speed	25 GbE	10 GbE

```

mjkkhtri@node0:~$ lscpu | head -20 # CPU model, cores, threads
Architecture: x86_64
CPU op-mode(s): 32-bit, 64-bit
Byte Order: Little Endian
Address sizes: 46 bits physical, 48 bits virtual
CPU(s): 40
On-line CPU(s) list: 0-39
Thread(s) per core: 2
Core(s) per socket: 10
Socket(s): 2
NUMA node(s): 2
Vendor ID: GenuineIntel
CPU family: 6
Model: 85
Model name: Intel(R) Xeon(R) Silver 4114 CPU @ 2.20GHz
Stepping: 4
CPU MHz: 800.181
CPU max MHz: 3000.0000
CPU min MHz: 800.0000
BogoMIPS: 4400.00
Virtualization: VT-x
mjkkhtri@node0:~$ cat /proc/cpuinfo | grep "model name" | head -1
model name      : Intel(R) Xeon(R) Silver 4114 CPU @ 2.20GHz
mjkkhtri@node0:~$ free -h # RAM
              total        used        free      shared  buff/cache   available
Mem:          187Gi        772Mi        186Gi         1.0Mi        194Mi        185Gi
Swap:          3.0Gi           0B          3.0Gi
mjkkhtri@node0:~$ lspci | grep -i net # NIC model - check for SR-IOV
3b:00.0 Ethernet controller: Intel Corporation Ethernet Controller X550 (rev 01)
3b:00.1 Ethernet controller: Intel Corporation Ethernet Controller X550 (rev 01)
5e:00.0 Ethernet controller: Intel Corporation Ethernet Controller X710 for 10GbE SFP+ (rev 01)
5e:00.1 Ethernet controller: Intel Corporation Ethernet Controller X710 for 10GbE SFP+ (rev 01)
mjkkhtri@node0:~$ ethtool -i <interface> # driver name (e.g., i40e, ice, ixg
be = SR-IOV capable)
-bash: syntax error near unexpected token `newline'
mjkkhtri@node0:~$ uname -r # current kernel
5.4.0-212-manjaro

```

Figure 1: L0 host hardware: Xeon Silver 4114, 46-bit physical addressing, approximately 187 GiB visible memory, and Intel X550/X710 network devices.

2.2. Software Stack

I built the software stack from the authors’ public HyperTurtle repository, which contains the modified Linux kernel, QEMU tree, and hyperupcall implementations. The L0 host ran Ubuntu 20.04.6 with the HyperTurtle Linux 5.16.0+ kernel and HyperTurtle QEMU 7.2.0. libbpf was built from the HyperTurtle Linux tree, with the QEMU source modified accordingly. The L1 guest ran Ubuntu 20.04.6 with the same kernel, 12 vCPUs, and 64 GiB of RAM. A 9p shared folder mounted the artifact tree from L0 into L1, allowing kernel sources, headers, and scripts to be shared without duplication. L2 guests ran as Kata Containers 3.2.0 with the bundled Kata guest kernel 6.1.38, with the active hypervisor configuration adjusted as needed during setup and evaluation.

Steps not covered by the artifact. Beyond a single L1 launch script (`run_virt_host`) and manual steps for attaching a networking hyperupcall, the repository provides no evaluation infrastructure. All benchmark drivers, measurement harnesses, Kata startup scripts, NGINX/Memcached/Redis load configurations, and profiling pipelines were written from scratch. Several code-level fixes were also required before any experiment could run:

- **Missing QEMU_CR3 injection:** the guest page-table root was not being exported into the shared eBPF state, so the EPT bypass could not resolve mappings correctly.
- **MAPS_DEV_OFFSET incorrect:** the value was 5 in `kvm_host.h` but must be 1. With the wrong value all shared maps were NULL and the hyperupcall was non-functional.
- **Hot-path `pr_err` calls:** five diagnostic prints in `arch/x86/kvm/x86.c` fired on every shared-memory EPT page fault, flooding the serial console and extending container boot times to over 23 s. All were commented out.

- **Exporter race during boot:** the original `ept_fault.guest` loop disabled the bypass during Docker/Kata process transitions. The fix used a temporary sticky-mode workaround together with a split create/start boot protocol.
- **Missing memory preallocation:** the final EPT path required preallocated guest memory so the exporter could populate the shared maps before the L2 guest started faulting pages.
- **Broken firewall sources:** the `packet_filter` sources in the repository snapshot had type errors and could not be built, so they were replaced with a minimal stateless XDP filter.
- **CPU pinning:** the HyperTurtle profiling path samples a fixed host CPU, so the hot outer L1 QEMU thread was pinned to host CPU 5 with `taskset` before each profiling run.

2.3. VM Configuration and Boot Parameters

Both L0 and L1 were booted with `idle=poll` and `nopti`. On L0, SMT was disabled, nested KVM was enabled, `ple_gap` was set to 0, and `vhost-net` zero-copy transmit was enabled. The `idle=poll` flag on L1 proved critical: without it, EPT fault latency appeared artificially low and inconsistent with the paper’s numbers. For the final EPT runs, the Kata configuration also used synchronous fault handling and guest-memory preallocation so that the exporter and bypass maps were populated before the L2 guest booted.

For networking, L1 was booted without `intel_iommu=on`. Including it caused DMAR faults on the virtio NIC path even without any hyperupcall attached, which invalidated all early networking measurements and took some time to diagnose.

For profiling, Kata’s default `cpu_features="pmu=off"` was changed to `pmu=on`, as the default configuration exposed no performance counters inside L2.

3. Methodology

3.1. EPT

The EPT latency microbenchmark allocated anonymous memory with `mmap` and touched each 4 KiB page once, measuring per-fault latency via `rdtsc`. For the Figure 9a result, the benchmark ran inside an L2 Kata container with the `ept_fault` hyperupcall loaded, the exporter started first, and `kvm-intel` reloaded so that the shared PCI-backed maps were discovered correctly. The non-nested reference run was collected directly inside L1.

For container startup, the protocol split launch into `docker create`, a short wait for the exporter to discover the L2 VMM and populate the maps, and then `docker start`. Six container images were tested (Alpine, Ubuntu, Java, JavaScript, Python, and a persistent-data variant), and startup time was measured until the containerised application responded over the network.

3.2. Networking

The networking evaluation targeted Table 7, Figures 12a/12b, and Figure 13. The benchmark datapath ran over L1 `enp1s0f0` through L0 tap device `tap1` to bridge `br0` at `10.10.1.0/24`. A key early finding was that HyperTurtle’s QEMU networking code resolves a *guest netdev index* rather than a Linux *ifindex*, and the correct value for the benchmark NIC was netdev index 1. Four hyperupcall variants were validated and benchmarked: `pass`, `rate_limiter`, `tcp_top`, and a stateless XDP firewall. Stale `clsact` qdiscs and orphaned XDP links were removed before each run to prevent attachment failures.

For NGINX (Figure 12a), a tiny-response benchmark proved insensitive to networking differences and was replaced with a 64 MiB static-file workload. For Memcached (Figure 13), the networking sweep used the custom benchmark driver developed for this reproduction to generate throughput-versus-p99 curves under the patched firewall. Absolute throughput values therefore reflect the chosen client, and the relative ordering across configurations is the primary comparison point.

3.3. Profiling

The profiling evaluation targeted Table 4 / Figure 9b and Figure 14. Three setup steps were required: exposing PMU counters inside Kata, building `mutilate` on L0 for Memcached load generation, and pinning the hot outer L1 QEMU thread to host CPU 5 because the HyperTurtle profiling path samples a fixed host CPU. Conventional L1 profiling was driven by a small helper based on `perf_event_open()`.

For Table 4 / Figure 9b, the $L0 \rightarrow L1$ and $HT \rightarrow L2$ entries were measured with the L0 `function_graph` tracer on the sampled CPU, while the $L1 \rightarrow L2$ path was reconstructed from L0 KVM tracepoint bursts. The $L1 \rightarrow L1$ case was not measured reliably because L1-local tracing misses time spent in L0 during the repeated world switches. For Figure 14, Memcached ran inside a Kata container and was loaded with `mutilate` using the Facebook ETC workload distribution. All runs used Nested-VirtIO networking, capping throughput at roughly 21 Kops/s instead of the paper’s 80 Kops/s. Three lanes were compared at 1 kHz and 4 kHz: no profiling, conventional L1 profiling, and HyperTurtle profiling.

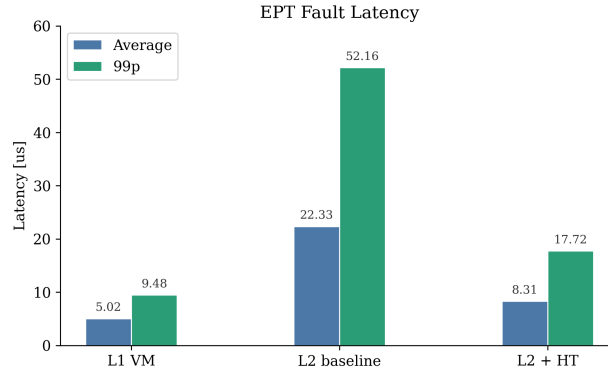
4. Results

4.1. EPT Fault Handling

HyperTurtle matches the paper’s main EPT result. As shown in Table 2 and Figure 2, the nested baseline measured $23.43 \mu\text{s}$ median fault latency, while HyperTurtle reduced that to $5.55 \mu\text{s}$, a $4.22\times$ speedup. The HyperTurtle median is within $0.92 \mu\text{s}$ of the non-nested L1 reference at $4.63 \mu\text{s}$, showing that the bypass reaches near-native latency under correct initialization.

Table 2: Final EPT fault microbenchmark results.

Configuration	Average (μs)	Median (μs)	99p (μs)	99.9p (μs)
L1 VM (non-nested)	5.02	4.63	9.48	46.75
L2 VM baseline	22.33	23.43	52.16	73.67
L2 VM + HyperTurtle	8.31	5.55	17.72	988.29

**Figure 2:** EPT fault latency. HyperTurtle reduces nested L2 fault latency to near-native L1 levels.

```

added device
got hypercall nr 17, args: 0 5389127680 1 1
user ptr: 0x7f830640000
create: i: 0, mem-slot: 14 slot size: 1000 start addr: fc000000 ram: 0x7f830640000
kvm_set_phys_mem: skip export for mem-slot 14 because as_id=1
added device
got hypercall nr 17, args: 0 5389791232 1 1
user ptr: 0x7f83063f000
create: i: 0, mem-slot: 15 slot size: 1000 start addr: fc000000 ram: 0x7f83063f000
kvm_set_phys_mem: skip export for mem-slot 15 because as_id=1
added device
got hypercall nr 17, args: 0 5389766556 1 1
user ptr: 0x7f83063b000
create: i: 0, mem-slot: 16 slot size: 4000 start addr: fc000000 ram: 0x7f83063b000
kvm_set_phys_mem: skip export for mem-slot 16 because as_id=1
added device
got hypercall nr 17, args: 0 5389058048 1 1
user ptr: 0x7f830633000
create: i: 0, mem-slot: 17 slot size: 8000 start addr: fc200000 ram: 0x7f830633000
kvm_set_phys_mem: skip export for mem-slot 17 because as_id=1
added device

pfn: 0x8180000000141379
Calling hypercall 17 with args 0 5389127680
hypercall 17 returned 1
Opening /sys/bus/pci/devices/0000:04:00.0/resource2
Mapping 4000 Bytes
pfn: 0x818000000014141b
Calling hypercall 17 with args 0 5389791232
hypercall 17 returned 4
Opening /sys/bus/pci/devices/0000:05:00.0/resource2
Mapping 4000 Bytes
pfn: 0x8180000000141415
Calling hypercall 17 with args 0 5389766556
hypercall 17 returned 5
Opening /sys/bus/pci/devices/0000:06:00.0/resource2
Mapping 4000 Bytes
pfn: 0x8180000000141368
Calling hypercall 17 with args 0 5389058048
hypercall 17 returned 6
Opening /sys/bus/pci/devices/0000:07:00.0/resource2
Mapping 4000 Bytes
got pfn: 0x7f0c7e2d0000 0x7f0c7e479000
starting loop
000: scanning /proc/1007/maps
L1 memslot[0]: base_gfn=0 npages=524288 userspace_addr=0x7fb3fbff000
000: enabled bypass for pid=1007

root@ubuntu:~# IMG="launch_bash_ubuntu"
root@ubuntu:~# NAME="microbench_test"
root@ubuntu:~# sudo docker rm -f $NAME 2>/dev/null
root@ubuntu:~# sudo docker create --name $NAME --runtime io.containerd.kata.v2 \
> -v /home/ubuntu/shared_folder/benchmarks:/bench $IMG /bin/bash -c "sleep infinity"
174cb452e12e39295063279499a07abb8becc90071bc39648cc71301439
root@ubuntu:~# sleep 2
root@ubuntu:~# sudo docker start $NAME
microbench_test
root@ubuntu:~# sudo docker exec -e RESULTS_DIR=/bench/results $NAME /bench/ept_fault_bench 10 12 ht_b
EPT Fault Latency Benchmark
Region: 10 MiB (2560 pages)
TSC freq: 2200.00 MHz
Measuring 2560 page faults...
Raw data saved to /bench/results/ept_fault_latencies_12_ht_10miB_10MiB.csv
=== EPT Fault Latency Results ===
Pages faulted: 2560
Average: 8.31 us
Median: 5.55 us
99th %ile: 17.72 us
99.9th %ile: 988.29 us
Min: 1.50 us
Max: 1416.88 us
root@ubuntu:~#

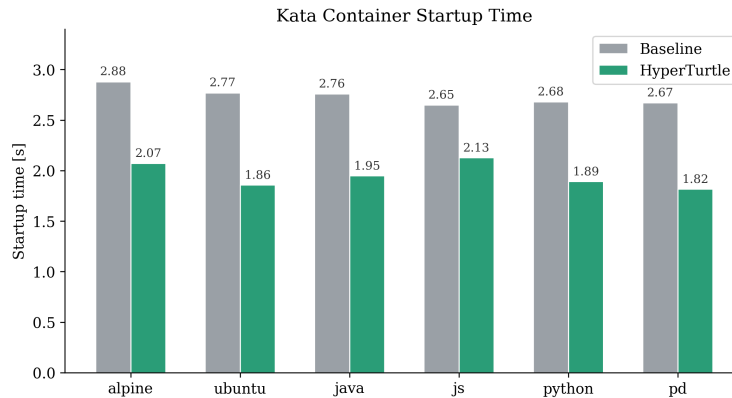
```

Figure 3: Terminal output from the final EPT benchmark run. The left pane shows the exporter discovering the L2 QEMU process and populating the shared maps. The right pane shows the benchmark result (median 5.55 μs , 99p 17.72 μs).

HyperTurtle also reduced container boot time across all six images (Table 3 and Figure 4), averaging 29% faster and ranging from 20% to 33%, reflecting the reduced EPT fault overhead during the guest kernel’s memory initialisation phase.

Table 3: Final Kata startup times with the working EPT bypass path.

Image	Baseline (s)	HyperTurtle (s)	Speedup
alpine	2.88	2.072	28%
ubuntu	2.77	1.857	33%
java	2.76	1.949	29%
js	2.65	2.127	20%
python	2.68	1.893	29%
pd	2.67	1.818	32%
Average	2.74	1.95	29%

**Figure 4:** Kata container startup time by image. HyperTurtle reduces boot time by 20–33% across all six images.

4.2. Networking

The networking results reproduce the paper’s main ordering across configurations: HyperTurtle stays close to the fast path and substantially outperforms the nested baseline on network-sensitive workloads. Note that the paper used a 25 GbE ConnectX-4 NIC whereas this setup is limited to 10 GbE, so absolute throughput values are not directly comparable and the observed gains are likely conservative.

The clearest application-level result is the large-file NGINX benchmark (Figure 12a equivalent). As shown in Figure 5, when serving a 64 MiB static file, the nested baseline achieved 1000.75 MiB/s while plain HyperTurtle reached 1812.38 MiB/s. The added network functions remained close to that fast path: `rate_limiter` at 1641.41 MiB/s and `tcp_top` at 1706.01 MiB/s. Figure 6 confirms the `rate_limiter` hyperupcall was active and passing traffic at approximately 20 Gbit/s. By contrast, the Redis GET benchmark corresponding to Figure 12b was largely flat on this setup: all

four rows remained between 436 and 446 MiB/s, so Redis did not expose a strong nested penalty here.

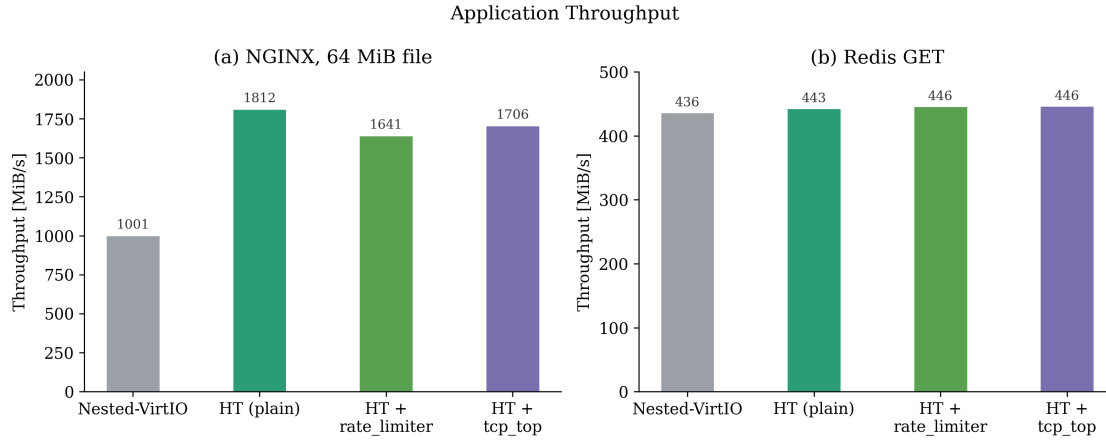


Figure 5: Application throughput. (a) NGINX: HyperTurtle stays well above the nested baseline even with active network functions. **(b) Redis:** all four configurations are close, making Redis an important negative result rather than a strong reproduction of the paper’s Figure 12b gap.

```

root@ubuntu:/home/ubuntu/shared_folder/hyperupcalls/network/pass# echo '==== clear dmesg [2:00:36.44/4]
e limiter reverse ====='
==== clear dmesg and run rate limiter reverse =====
root@ubuntu:/home/ubuntu/shared_folder/hyperupcalls/network/pass# dmesg -c 2>/dev/null || true
root@ubuntu:/home/ubuntu/shared_folder/hyperupcalls/network/pass# sleep 1
root@ubuntu:/home/ubuntu/shared_folder/hyperupcalls/network/pass# iperf3 -c 10.10.1.1 -R -t 5
Connecting to host 10.10.1.1, port 5201
Reverse mode, remote host 10.10.1.1 is sending
[ 5] local 10.10.1.2 port 35886 connected to 10.10.1.1 port 5201
[ ID] Interval      Transfer    Bitrate
[ 5] 0.00-1.00 sec  1.96 GBytes 16.9 Gbits/sec
[ 5] 1.00-2.00 sec  1.99 GBytes 17.1 Gbits/sec
[ 5] 2.00-3.00 sec  2.61 GBytes 22.4 Gbits/sec
[ 5] 3.00-4.00 sec  2.63 GBytes 22.6 Gbits/sec
[ 5] 4.00-5.00 sec  2.64 GBytes 22.7 Gbits/sec
- - - - -
[ ID] Interval      Transfer    Bitrate      Retr      sender
[ 5] 0.00-5.00 sec  11.8 GBytes 20.3 Gbits/sec  0         receiver
[ 5] 0.00-5.00 sec  11.8 GBytes 20.3 Gbits/sec
iperf Done.

```

Figure 6: iperf3 reverse throughput with the rate_limiter hyperupcall attached, confirming the function was active and passing traffic at approximately 20 Gbit/s.

The Memcached-under-firewall experiment best captures the Figure 13 trend. Figure 7 and Table 4 show that at the 500 μ s p99 target, HyperTurtle with the patched firewall sustained substantially more throughput than the nested baseline with firewall, a 26% advantage at the nearest comparable operating point. A no-firewall plain HyperTurtle run reached a very similar point, confirming that the firewall itself imposed only a modest additional cost, consistent with the paper’s claim. Figure 8 confirms the firewall hyperupcall was successfully loaded during these runs.

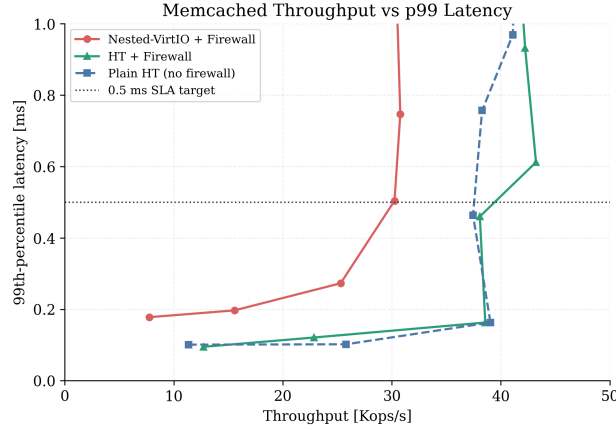


Figure 7: Memcached throughput versus 99th-percentile latency. HyperTurtle with firewall support sustains substantially more throughput than the nested baseline near the 500 μ s target.

```

root@ubuntu:/home/ubuntu/shared_folder/hypercalls/network/packet_filter# ./packet_filter.guest
ptr.array: 0x1
ptr.array: 0x7ffff7fb000
Opened pagemap 3
pfn: 0x818000000016c4df
pfn: 0x818000000015071a
idx: 0, va: 0x7ffff7fc6000 pa: 0x15071a000
pfn: 0x818000000016c4de
idx: 1, va: 0x7ffff7fc7000 pa: 0x16c4de000
pfn: 0x8180000000196ca5
idx: 2, va: 0x7ffff7fc8000 pa: 0x196ca5000
Calling hypercall 13 with args 6112014336 11880
Hypercall 13 returned 0
pfn: 0x818000000015071a
Calling hypercall 15 with args 0 5644591104 0 1
Hypercall 0 returned 1
packet_filter running; Ctrl+C to stop

```

Figure 8: Terminal output confirming the firewall hyperupcall loaded and linked successfully during the Memcached experiments.

Table 4: Networking operating-point comparison near the 500 μ s p99 target.

Configuration	Throughput (Kops/s)	p99 (μ s)	Relative
Nested-VirtIO + Firewall	30.25	504	1.00×
HyperTurtle + Firewall	38.07	460	1.26×
Plain HyperTurtle (no firewall)	37.45	464	1.24×

Table 5 shows the Table 7 proxy: the nested baseline reached 10.61 Gb/s while HyperTurtle + Pass and HyperTurtle + Firewall reached 19.40 and 18.73 Gb/s respectively, capturing the correct ordering. A Direct Assignment row was not included because VFIO device assignment could not be configured under the default Kata QEMU setup.

Table 5: Table 7 proxy from the networking experiments.

Configuration	Throughput (Gb/s)	Avg. latency (μ s)
Nested-VirtIO	10.61	N/A
HyperTurtle + Pass	19.40	59.87
HyperTurtle + Firewall	18.73	59.80

4.3. Profiling

The profiling path worked correctly once the hot outer L1 QEMU thread was pinned to host CPU 5: the `rbp_traces` map advanced at 1,000.3 Hz during live Kata execution, confirming that HyperTurtle profiling was active. Three of the four Table 4 / Figure 9b entries were measured. As shown in Figure 9, L0 $\rightarrow$ L1 measured 4.93μ s against the paper’s 5.66μ s, HT $\rightarrow$ L2 measured 7.02μ s against the paper’s implied $\approx 8.5 \mu$ s, and L1 $\rightarrow$ L2 measured approximately 75μ s against the paper’s 60.68μ s, giving a measured L1 $\rightarrow$ L2 to HT $\rightarrow$ L2 speedup of $10.7\times$. The remaining L1 $\rightarrow$ L1 entry was not measured reliably.

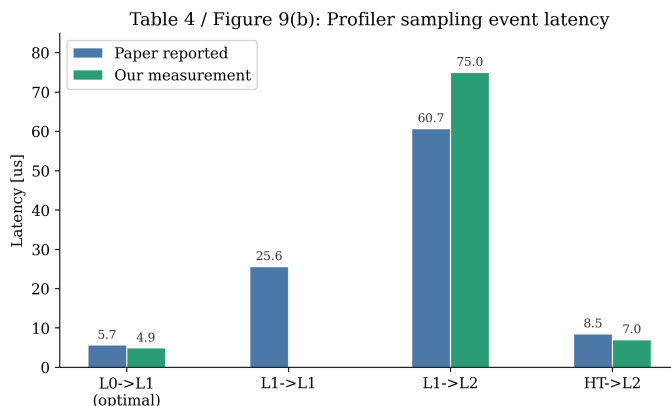


Figure 9: Profiler sampling-event handle time (Table 4 / Figure 9b). Three of the four configurations were measured on this stack: L0 $\rightarrow$ L1 at 4.93μ s, L1 $\rightarrow$ L2 at approximately 75μ s, and HT $\rightarrow$ L2 at 7.02μ s. Only the L1 $\rightarrow$ L1 entry remains unmeasured.

The 1 kHz Memcached sweep (Figure 10) shows a clear separation between the three modes. HyperTurtle profiling stays within $5\text{--}17 \mu$ s of the no-profiling baseline across the $2\text{--}20$ Kops/s range, while conventional L1 profiling adds $40\text{--}95 \mu$ s. At 20 Kops/s, the p99 latency rises from 258.5μ s with no profiling to 275.0μ s with HyperTurtle and 353.1μ s with conventional L1 profiling. A 4 MiB GUPS-like proxy workload also showed no visible throughput overhead at 1 kHz in any mode, with all three configurations measuring approximately 310 Mops across three runs.

Figure 14: p99 latencies of Memcached get requests during profiling

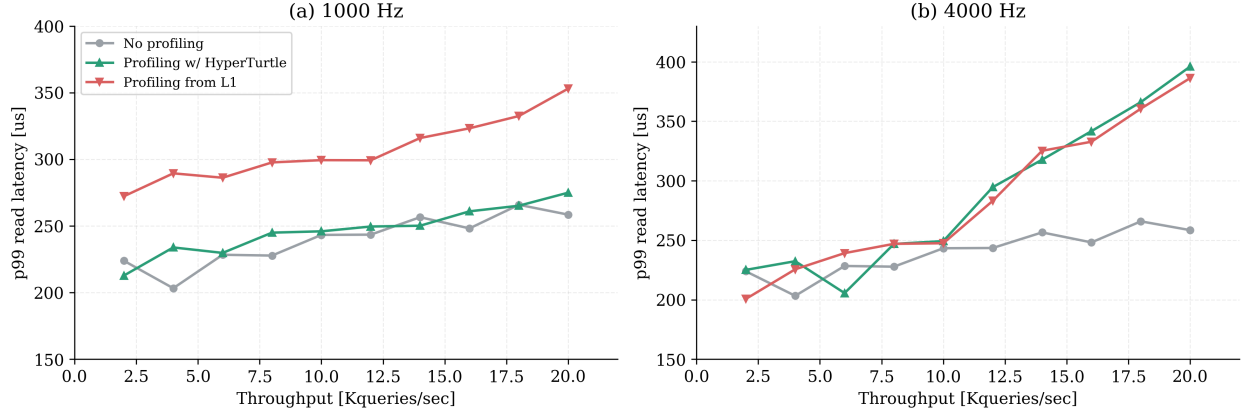


Figure 10: Memcached p99 read latency versus throughput under profiling, measured with `mutilate` using the Facebook ETC workload distribution. **(a) 1 kHz:** HyperTurtle profiling adds only 5–17 μs over the no-profiling baseline while conventional L1 profiling adds 40–95 μs . **(b) 4 kHz:** HyperTurtle and L1 profiling are nearly indistinguishable on this stack, likely because Nested-VirtIO networking caps throughput at ≈ 21 Kops/s.

At 4 kHz, HyperTurtle and L1 profiling are nearly indistinguishable on this stack. The most likely cause is the Nested-VirtIO throughput cap at approximately 21 Kops/s, well below the load range reached by the paper with Direct-Assignment.

5. Conclusion

This reproduction validates the paper’s three main claims. On EPT, HyperTurtle reduced median nested fault latency from 23.43 μs to 5.55 μs (4.22 $\times$) and improved Kata startup time by 20–33% across all six images. On networking, HyperTurtle stayed close to the fast path and substantially outperformed the nested baseline on NGINX and Memcached workloads. On profiling, HyperTurtle added only 5–17 μs to Memcached p99 latency at 1 kHz versus 40–95 μs for conventional L1 profiling, and the measured HT $\rightarrow$ L2 handle time of 7.02 μs against 75 μs for L1 $\rightarrow$ L2 gives a 10.7 $\times$ speedup consistent with the paper’s result.